

FUSE Package Author Guide

Authoring and converting FUSE packages

Contents

1. Package Author Guide
2. JSON Schema Reference
3. Converter
4. External Editor
5. Migrating From Legacy Mods
6. Architecture

Package Author Guide

This guide is the short public authoring contract for FUSE packages. The full schema is documented in `../schemas/FUSE_JSON_SCHEMA.md` and `schemas/fuse-mod.schema.json`.

File Shape

Use one or more `*.fuse.json` files inside a FUSE package folder. Each file should keep one clear concern where practical: graph, scenery, industries, progressions, audio, map tiles, or asset metadata.

Every file should include:

- `schemaVersion`
- `id`
- `name`
- `modVersion`

`author` may be blank if the source package does not provide one.

Id Rules

- Package ids should be stable across releases.
- Object ids should be stable and human-readable.
- Do not reuse the same object id for different objects in one package.
- Do not rename an id just to change display text.
- Keep display names in `name`; keep identity in `id`.

Player-Selectable Features

Use top-level settings plus `featureRules` when one package should offer optional track, scenery, industries, loaders, or other authored sections. The Tile Editor's **Options** workspace creates on/off, choice, and slider settings, lets you select the exact objects controlled by each option, and writes both dictionaries together.

Feature settings must be marked `reloadRequired`; the Editor does this automatically. A false rule omits only the targets listed by that rule from the runtime definition. It does not delete those objects from the package file. Every target must be authored in the same definition, and an industry-component target uses `industryId/componentId`. See the schema guide for the full target list and JSON example.

RailLoader output has no equivalent contract. The Editor therefore disables this workspace in legacy mode instead of creating a lossy or misleading export.

Dependencies

Use package metadata for required package ordering and dependencies:

- `FuseLoadPriority`
- `FuseRequires` for hard FUSE data-package dependencies
- `FuseLoadAfter`
- `FuseLoadBefore`
- `FuseConflictsWith` for explicit package incompatibilities, with optional `NotBefore/NotAfter` bounds

- normal UMM Requirements/LoadAfter when a UMM code mod requires FUSE or another UMM mod

Use `mixinto` when converting legacy conditional mixin files. Missing `mixinto` requirements or matching `mixinto.conflictswith` references skip only that fragment, not the whole stack.

Optional References

References that may not exist on every route should be treated as optional by schema/runtime design. Missing optional references should log an info-level skip, not fault an unrelated package.

Hard references should identify:

- package id
- source object id
- source field
- target kind
- target id

Graph Data

Track graph data is based on node, segment, and span ids. Spans must point at valid segment ids and valid segment ends. FUSE can reference base-game graph objects at runtime, but converted packages should not invent ids that cannot exist in the runtime graph.

Use `/fuse.dumpgraph` and `/fuse.dumpruntimegraph` to inspect graph state.

Operations Data

Supported industry component types include:

- loader
- unloader
- formulaic
- repairTrack
- teamTrack
- interchange
- interchangedLoader
- interchangedUnloader
- teleportLoading
- progression
- passengerStop
- fully-qualified custom `IndustryComponent` types from loaded assemblies

Custom components can use `fields` for reflection-bound values. The custom component assembly must be installed separately.

World Data

Use `world.scenery` for asset-pack objects. Use `world.sceneClones` for base-game scene objects. Use `world.mapMasks` for terrain flattening, tree cutting, height masks, and mask modifiers. Use `world.splines` for roads, rivers, trestles, and related spline builders.

Packages with a map declaration are treated as complete replacement worlds: `map.suppressBaseWorld` defaults to `true`. FUSE keeps Railroader's required scene managers but removes the stock track graph and suppresses the stock operations, scenery, map labels, signs, setups, progression, and CTC content before applying the selected package. Set `suppressBaseWorld` to `false` only for a map that intentionally overlays custom terrain on Bushnell/Whittier.

Asset pack objects should keep their real asset identifiers. Do not alias to unrelated assets if the correct pack exists.

Asset stores and definition overrides

Declare normal asset-pack roots with `FuseAssetPacks`. A runtime store is identified by `Catalog.json`; `Bundle` is optional for a definitions-only catalog whose assets are supplied by another store. FUSE reports an actual missing bundle if an asset from that store is requested.

For the old `AssetLoader` pattern where a `Definitions.json` file replaces the definitions of an existing store (rolling-stock and tender swaps), prefer an explicit native manifest entry:

```
{
  "Requirements": ["FUSE"],
  "FuseDefinitionOverrides": [
    {
      "StoreIdentifier": "fm-flatcar03",
      "Path": "DefinitionOverrides/fm-flatcar03/Definitions.json"
    }
  ]
}
```

The path must stay inside the package. An object entry names the exact existing store id; a string path infers the id from its parent folder. FUSE also detects `AssetLoader`'s legacy immediate-child convention automatically, but native packages should be explicit. If two packages target the same exact store, FUSE chooses deterministically and reports both source files.

New packages should require FUSE, not `AssetLoader`. The installer's data-only `AssetLoader` alias exists only for old manifests.

Diagnostics

Use these commands while authoring:

- `/fuse.report`
- `/fuse.loaded`
- `/fuse.conflicts`
- `/fuse.graph`
- `/fuse.operations`
- `/fuse.progressions`
- `/fuse.assets`

- `/fuse.dumpgraph`
- `/fuse.dumpruntimegraph`
- `/fuse.dumpmandelas`

Warnings should name package id, operation, object id, and field whenever possible.

Malformed JSON and validation faults are isolated to the affected definition. `/fuse.report` and `/fuse.report.json` include the absolute folder/file, JSON path, line/column when available, and a suggested action. Treat a report with a faulted package as an authoring failure even if unrelated packages still work.

For task-based examples, see Authoring Recipes.

JSON Schema Reference

This folder defines the JSON side of the FUSE mod format.

- `fuse-mod.schema.json` is the authoritative JSON Schema for hand-authored and editor-exported `.json` files.
- `fuse-mod.example.json` is a compact example that exercises track, spans, areas, industry ordering, built-in and custom industry components, loaders, turntables, roundhouses, stations, scenery, spawn points, span-anchored scenery, splineys, water surfaces, telegraph poles, labels, speed signs, masks, suppressions, map tiles, scene clones, world removals, progression data, audio definitions, mixinto metadata, package settings, and editor state.
- `umm-info.schema.json` documents the Unity Mod Manager Info.json shape FUSE expects for API mods, data packages, and asset-pack packages.
- `umm-info.example.json` is a data-only map package manifest that depends on FUSE.

Design Choices

The schema uses one document for the whole mod. The shipped binary `.bson` file should use the same logical model, even if the serializer writes a compact representation internally.

Top-level object groups:

- `tracks`: nodes, segments, spans, areas, and optional removals for deleting base-game track objects.
- `operations`: loads, industries, loaders, turntables, and passenger stations.
- `world`: scenery, spawn points, splineys, polygonal water surfaces, telegraph poles, map labels, map masks, map tile overlays, scene clones, and optional removals for base scene objects.
- `progression`: progression trees and map features.

Map declarations are complete replacement worlds by default. `map.suppressBaseWorld` defaults to `true`, which removes the stock Bushnell/Whittier graph and suppresses its operations, scenery, labels, signs, setup, progression, and CTC content before the selected package applies. Set it to `false` only when the package deliberately uses custom terrain as an overlay on the stock world.

- `settings`: package-declared UI settings with user, profile, or server scope. Runtime values are stored under `LocalLow/FUSE`, not written into package folders.
- `editor`: optional editor-only state that FUSE can ignore at runtime.
- `extensions`: optional namespaced third-party data.

`mixinto` is optional metadata for converted RailLoader/Strange Customs conditional mixin files. The converted file remains a normal FUSE fragment, but FUSE checks `mixinto.requires[]` and `mixinto.conflictsWith[]` immediately before runtime apply. A missing/out-of-range requirement or a matching conflict skips that fragment without faulting the rest of the package.

```
{
  "mixinto": {
    "target": "game-graph",
    "sourceFile": "CNRI_FoxysLimestonePatch.json",
    "requires": [
      { "id": "CaptainFoxy.NantahalaLime", "notBefore": "1.1" }
    ],
    "conflictsWith": [
      { "id": "Some.Incompatible.Route", "notAfter": "3.0" }
    ]
  }
}
```

A converted FUSE package id ending in .FUSE also satisfies the original legacy id, so CaptainFoxy.NantahalaLime.FUSE satisfies a requirement for CaptainFoxy.NantahalaLime.

Package settings are declared in a top-level settings dictionary and rendered automatically in the FUSE Settings page. Supported type values are bool, enum, number, path, color, and text. Supported scope values are:

- user: local player value, shared by all profiles.
- profile: value keyed to the active FUSE mod-set/profile.
- server: value keyed to the active profile fingerprint for server-style shared configuration.

Enum values are treated as exact strings. FUSE does not normalize or rename setting values.

```
{
  "settings": {
    "yardDetailLevel": {
      "type": "enum",
      "label": "Yard Detail",
      "scope": "profile",
      "values": ["low", "standard", "high"],
      "default": "standard",
      "reloadRequired": true
    }
  }
}
```

Native modular packages can connect a setting to authored content with a top-level featureRules dictionary. When a rule does not match, FUSE omits only its named targets from the runtime copy of the definition. The source JSON stays intact, so changing the option and reloading restores the objects.

```

{
  "settings": {
    "enableExtraYard": {
      "type": "bool",
      "label": "Extra Yard Tracks",
      "scope": "profile",
      "default": true,
      "reloadRequired": true
    }
  },
  "featureRules": {
    "extraYard": {
      "setting": "enableExtraYard",
      "operator": "equals",
      "value": true,
      "targets": {
        "trackNodes": ["yard:n:4", "yard:n:5"],
        "trackSegments": ["yard:s:extra"],
        "trackSpans": ["yard:span:extra"],
        "scenery": ["yard:prop:bumper"]
      }
    }
  }
}

```

Operators are `equals`, `notEquals`, `greaterThan`, `greaterThanOrEqualTo`, `lessThan`, and `lessThanOrEqualTo`; ordering operators require a number setting. Target keys cover track, operations, world, progression, and audio dictionaries. Industry components use `industryId/componentId`. Every target must be authored in the same definition, which prevents an option from accidentally deleting base-game or dependency objects. Rules are native FUSE data and are not projected into RailLoader JSON.

All editable game objects are stored in dictionaries keyed by object ID. The ID is not repeated inside the object body. This keeps editor updates simple: replacing `tracks.nodes["murphy:n:001"]` updates exactly one object without array searches or ID duplication.

`schemaVersion` is the FUSE data format version and drives migrations. Use the string form, for example `"1.0"`. Integer `1` remains accepted by the runtime for v1.0 packages already in the wild. `modVersion` is the author's release version and should not be used for schema migration decisions.

Versioning And Deprecation

FUSE migrations run version by version on load. Unknown future schema versions are logged as warnings and loaded on a best-effort basis instead of failing immediately.

When a field is renamed, FUSE keeps the old field readable for one minor version, logs one deprecation warning per package when it migrates that field, and removes the old field in the following minor version. For example, scenery asset keys were renamed from `model` to `assetIdentifier`; v1.0 packages with only `model` still load, and FUSE fills `assetIdentifier` in memory.

Naming And Legacy Aliases

The public mod/API name is FUSE. New packages should use `.FUSE` folder/package suffixes, `FuseDataFile/FuseDataFiles` in `Info.json`, and `.fuse.json` data files.

Compatibility aliases are intentional and narrow:

- A dependency or mixinto requirement ending in `.RAIL` can satisfy the same package without that old suffix. This is only to keep already-converted packages from faulting during the rename window.
- The literal asset-pack id `rail` can still appear inside Railroader `AssetReference` values. That string is Railroader's built-in asset-pack identifier, not the old `RAIL` mod name.

Do not author new public docs or examples with `RAIL` as the mod name.

Example Coverage

The shipped examples cover the public authoring cases:

Case	Example file	Notes
Route/data package	<code>fuse-mod.example.json</code>	Track, areas, spans, operations, world objects, progression, audio, and editor metadata.
UMM package manifest	<code>umm-info.example.json</code>	<code>FuseDataFiles</code> , load ordering, and <code>FuseAssetPacks</code> .
Asset pack package	<code>umm-info.example.json</code>	Use <code>FuseAssetPacks</code> to point at one or more package-relative asset pack roots.
Map tiles	<code>fuse-mod.example.json</code>	<code>world.mapTiles</code> overlays <code>tile_XXX_YYY.data</code> files for a <code>MapManager.directoryName</code> .
Audio pack	<code>fuse-mod.example.json</code>	<code>audio.whistles</code> , <code>audio.horns</code> , and <code>audio.bells</code> .
Industry component pack	<code>fuse-mod.example.json</code>	Built-in component types plus a fully-qualified custom component type with <code>fields</code> .
Load component pack	<code>fuse-mod.example.json</code>	<code>operations.loads.*.fields</code> can carry reflection-bound custom load data.
Package settings	<code>fuse-mod.example.json</code>	Top-level settings with <code>bool</code> , <code>enum</code> , <code>color</code> , <code>user/profile/server</code> scope, reload metadata, and object-level <code>featureRules</code> .
Mixinto	<code>fuse-mod.example.json</code>	<code>mixinto.target</code> , <code>mixinto.sourceFile</code> , conditional <code>requires</code> , and conditional <code>conflictsWith</code> .
Progression section	<code>fuse-mod.example.json</code>	Root <code>progression.sections[]</code> , delivery phases, unlock features, and interchange transfers.
Map mask	<code>fuse-mod.example.json</code>	<code>world.mapMasks</code> for terrain/object mask authoring.
Scene clone	<code>fuse-mod.example.json</code>	<code>world.sceneClones</code> for base-game object cloning/retargeting.

Case	Example file	Notes
Water surface	<code>fuse-mod.example.json</code>	<code>world.waterSurfaces</code> for FUSE-owned lake planes and optional stock-lake material reuse.
Span-anchored scenery	<code>fuse-mod.example.json</code>	<code>world.scenery.*.anchorSpanIds</code> .
Signals	Not included	Signals are deferred and should not be treated as supported yet.

The converter guide at `docs/FUSE_CONVERTER.md` covers drag/drop conversion, batch conversion, output reports, and warning classification.

Areas are defined under `tracks.areas` and can be used to group industries in the company window. `order` is a signed sort key that controls the display order of areas and industries; lower values appear earlier. Legacy conversions preserve source order values, including negative values, when the source file provides them:

```
{
  "tracks": {
    "areas": {
      "murphy:area:town": {
        "name": "Murphy",
        "order": 0,
        "position": { "x": 25, "y": 0, "z": 0 },
        "radius": 350,
        "spanIds": ["murphy:span:depot"]
      }
    }
  },
  "operations": {
    "industries": {
      "murphy:ind:depot": {
        "name": "Murphy Depot",
        "areaId": "murphy:area:town",
        "order": 1,
        "position": { "x": 12, "y": 0, "z": -9 }
      }
    }
  }
}
```

Vector values are always objects:

```
{ "x": 0, "y": 0, "z": 0 }
```

Track spans use two structured locations rather than Strange Customs style strings. Think of each location as an arrow measured from one segment end into the span. `Start/A` points from the segment's start toward its end; `End/B` points from the segment's end toward its start. The two endpoint arrows must face each other and the measured distance must stay within that segment.

```
{
  "upper": { "segmentId": "murphy:s:001", "distance": 10, "end": "Start" },
  "lower": { "segmentId": "murphy:s:001", "distance": 5, "end": "End" }
}
```

end is optional and defaults to `A/Start`. Set it to `B/End` when the distance or normalized value is measured from the segment's far end. On the same segment, one endpoint must use `Start/A` and the other must use `End/B`; FUSE rejects same-direction endpoints, crossed endpoints, zero-length spans, and distances outside the segment length.

Use `normalized` for editor-authored spans where possible. Use `distance` only when the exact distance along a runtime segment matters.

When translating legacy graph patches that delete existing base-game track objects, use `tracks.removals`. These values must be exact graph object IDs from the source game graph. Base-game track nodes usually look like short four-character alphanumeric IDs; do not use UI names such as siding, spur, or station names unless the source graph actually uses that string as the object ID.

```
{
  "tracks": {
    "removals": {
      "nodes": ["A1B2"],
      "segments": ["C3D4"],
      "spans": ["E5F6"]
    }
  }
}
```

FUSE-authored nodes, segments, and spans can use descriptive package-owned IDs such as `murphy:n:001`, but base-game removals must use the IDs that already exist in Railroader's live graph. FUSE applies span removals first, then segment removals, then node removals.

Legacy source files can also use `null` entries to delete world objects such as old roads or scenery. FUSE stores those as `world.removals`, using either FUSE IDs or full scene paths:

```
{
  "world": {
    "removals": {
      "scenery": ["World/Large Scenery/Murphy/Old Depot"],
      "splines": ["World/Large Scenery/Murphy/Old Road"],
      "mapLabels": [],
      "mapMasks": [],
      "telegraphPoles": [],
      "sceneClones": []
    }
  }
}
```

The converter maps legacy `null` entries from `scenery`, `splines`, `mandelas`, and `texts` into these arrays.

Progression can be authored either in the older keyed form under `progressions.<id>.sections` or in the flatter root `progression.sections[]` form. Root sections must set `id`; `progression.progressionId` selects the runtime `Progression.identifier`, and defaults to the package id when omitted. FUSE normalizes root sections into the existing Railroader progression system at load time.

Section unlock payloads such as `areasEnableOnUnlock`, `gameObjectsEnableOnUnlock`, `unlockIncludeIndustries`, `unlockExcludeIndustries`, `unlockIncludeIndustryComponents`, `trackGroupsEnableOnUnlock`, and `trackGroupsAvailableOnUnlock` are materialized through a synthetic FUSE `MapFeature` that is enabled when the section unlocks. This keeps narrative packages on the base game's progression path instead of inventing a parallel unlock system.

Declaring progressions.<id> for an id that already exists at runtime (for example the base career progression ewh) patches that progression in place: its sections are merged onto the live base progression instead of creating a parallel one. This is how a map mod's milestones ride the normal Company career.

progressions.<id>.enableFeaturesAtStart lists map features that are force-enabled every time that progression is configured in Company mode. It mirrors the game's own Progression.enableFeaturesAtStart — the lever the base career uses for wh-el / ewh-intch: on every load the game calls SetFeatureEnabled(feature, true) for each entry, so the enable is persisted into the save and self-heals existing saves. It never fires in sandbox (the game does not configure a progression there; sandbox is covered by initiallyEnabled). This is the correct way to make a mod's starting trunk present at career start. Note that initiallyEnabled on a map feature is sandbox-only and cannot do this. The field follows FuseStringPatch semantics: an array replaces the list, an object ({ "id": true }) merges per id — use the object form when patching a base-game progression so its own start features are preserved:

```
{
  "progression": {
    "progressions": {
      "ewh": {
        "enableFeaturesAtStart": { "My-Start-Trunk": true, "My-Start-Yard": true },
        "sections": { "...": {} }
      }
    }
  }
}
```

A feature granted through enableFeaturesAtStart must not also appear in any section's enableFeaturesOnAvailable / enableFeaturesOnUnlock / disableFeaturesOnUnlock arrays on that progression: the game rewrites those section-derived enables on every state update and would overwrite the start grant.

Section interchangeTransfers maps source interchange component ids to destination interchange component ids. FUSE materializes these as child InterchangeTransfer components on the runtime Section, matching the base game's section-completion waybill rewrite path. Null or blank destinations are kept as no-op legacy entries.

Progression delivery phases that contain deliveries should set industryComponentId when the target progression industry component is known. If omitted, FUSE can infer it when every delivery in the phase points at the same destinationIndustryId and that industry has exactly one ProgressionIndustryComponent.

Turntables generate deterministic pit node IDs at load time:

- operations.turntables["bryson"].subdivisions = 16
- generated pit nodes: bryson.pit.00 through bryson.pit.15

Track segments may reference those generated IDs directly. This keeps turntable-connected track stable across editor saves.

When translating legacy turntables that already have authored helper node IDs, set legacyIdentifier so FUSE reproduces the original NTurntableNode*, NRoundhouseNode*, and SRoundhouseSegment* naming pattern. Roundhouses can be generated from vanilla prefab aliases:

```

{
  "operations": {
    "turntables": {
      "murphy:turntable": {
        "position": { "x": 90, "y": 0, "z": 25 },
        "radius": 15,
        "subdivisions": 16,
        "legacyIdentifier": "MurphyTurntable",
        "roundhouse": {
          "stalls": 3,
          "startAngle": -18,
          "stallAngle": 12,
          "trackLength": 46,
          "startPrefab": "vanilla://roundhouseStart",
          "endPrefab": "vanilla://roundhouseEnd",
          "stallPrefab": "vanilla://roundhouseStall"
        },
        "visuals": {
          "pitAssetIdentifier": "rlw-ng-turntable-pit",
          "bridgeAssetIdentifier": "rlw-ng-turntable-bridge",
          "controllerType": "Toolshed.Turntables.HandTurntableController, Toolshed",
          "interactionRadius": 16
        }
      }
    }
  }
}

```

visuals is optional. If it is omitted, FUSE uses the vanilla turntable template and `TurntableController`. If it names custom pit/bridge scenery assets, FUSE creates a bare runtime `Track.Turntable`, loads the pit as a static visual, rotates the bridge visual with the runtime turntable angle, and attaches the optional `controllerType`. The controller type should be an assembly-qualified `MonoBehaviour` implementing `FUSE.API.IFuseTurntableController`.

Track segment `speedLimit` may be 0 through 80. Some legacy graphs use 0 for unrestricted or placeholder track speeds, and FUSE preserves that value during translation.

Track segments may also set optional companion-mod gauge metadata:

```

{
  "tracks": {
    "segments": {
      "murphy:s:ng-yard": {
        "startNodeId": "murphy:n:001",
        "endNodeId": "murphy:n:002",
        "style": "yard",
        "gauge": "Narrow"
      }
    }
  }
}

```

FUSE stores gauge for mods such as `NarrowGaugeMod`; base Railroader track geometry remains unchanged unless a companion mod acts on it. Native values are `Standard`, `Narrow`, `DualGauge`, `DualGauge_L`, `DualGauge_R`, and `DualGauge_T`; the additional legacy aliases accepted by the schema are read compatibility values, not recommended new output.

Asset and prefab references are URI strings:

- `vanilla://waterTower`
- `scenery://aspen-corner-drug`
- `path://scene/World/Large Scenery/Whittier/Coal Conveyor`
- `asset://example.route/depotSmall`
- `fuse://some-shared-catalog/object-id`

The current runtime directly understands `vanilla://`, `path://`, `scenery://`, and `empty://`. Other schemes may be reserved by tools or future loaders.

Scenery can optionally set `anchorSpanIds` for track-bound props such as railroad crossings. FUSE averages the referenced span centers, aligns the scenery to the average span tangent, then applies position and rotation as offsets:

```
{
  "world": {
    "scenery": {
      "murphy:crossing:depot": {
        "assetIdentifier": "scenery://crossing-board",
        "anchorSpanIds": ["murphy:span:depot"],
        "position": { "x": 0, "y": 0, "z": 0 },
        "rotation": { "x": 0, "y": 90, "z": 0 }
      }
    }
  }
}
```

`world.spawnPoints[]` registers `Railroader.Character.SpawnPoint` components. `name` and `position` are required; `rotation`, `radius`, and `priority` mirror the base-game component fields. Higher priority spawn points sort ahead of lower-priority entries in the game's spawn-point list.

`Spliney` type describes the physical spline family, not its material flavor:

- `river`: water spline, with one river behavior family.
- `road`: normal road spline. Use `style/profile` for dirt versus pavement.
- `terrainRoad`: terrain-carved road spline. Use `style/profile` for dirt versus pavement.
- `trestle`: auto-generated trestle spline.
- `objectLine`: repeats a scenery asset or safe scene-path prefab along a piecewise path. Use this for fence panels, retaining-wall sections, guardrails, hedges, and other rigid modules that must not be stretched into a road/river mesh. Set exactly one of `assetIdentifier` or `prefab`, then use `spacing`, `instanceScale`, `rotationOffset`, lateral/vertical offsets, optional terrain snapping/slope alignment, and the bounded `maximumInstances` safety limit.

Converted Strange Customs `FlowThingBuilder` data must inspect `style` and `profile`; entries with `style: "River"` or river profiles should be emitted as type: `"river"`, not as roads. When converting `FlowThingBuilder` entries, preserve the legacy default `offsetY: -0.1` if the source omits it; the field keeps road and river surfaces at the same vertical bias Strange Customs used.

Telegraph pole definitions use the existing Railroader telegraph pole and wire prefabs by default. `profile` selects the first vanilla pole prefab whose name contains that profile string. `polePrefab` and `wirePrefab` can override that with explicit prefab URIs.

`world.telegraphPoleMovements[]` adjusts existing base-game telegraph pole graph nodes by pole/node index. This is intentionally separate from `world.telegraphPoles`, which creates new pole sets. Use it for legacy TelegraphPoleMover style data:

```
{
  "world": {
    "telegraphPoleMovements": [
      {
        "poleIndices": [585, 583],
        "offset": { "x": 0, "y": 3, "z": 0 }
      }
    ]
  }
}
```

FUSE applies pole movements idempotently per package, so snapshot reapply does not stack the same offset repeatedly. Unloading the package restores the captured base pole positions, then reapplies any remaining package claims.

Industry components currently supported by the runtime include `loader`, `unloader`, `formulaic`, `repairTrack`, `teamTrack`, `interchange`, `interchangedLoader`, `interchangedUnloader`, `teleportLoading`, `progression`, and `passengerStop`. Formulaic components are attached directly to the industry object to match the base game component layout expectations; other component types get child objects. Fully-qualified custom IndustryComponent type names are accepted as compatibility surface when the owning assembly is loaded; FUSE reflectively binds common fields such as `load`, `convertedLoad`, `carLoadRate`, `carUnloadRate`, `loadRate`, `maxStorage`, `costPerUnit`, `working hours`, `fill percentage`, `title`, and `book reasons`. Custom components may also use fields for additional reflection-bound field/property values, so a separate component mod can expose data without requiring a FUSE code change. Legacy ConfusingSupplements shortcuts (`CaptiveConversionLoader`, `CaptiveConversionUnloader`, `Pay4Resource`, and `Empty`) are normalized to their fully-qualified `ConfusingSupplements.IndustryComponents.* runtime types`.

The converter emits canonical FUSE component type names. Legacy aliases such as `Model.Ops.IndustryLoader`, `Model.OpsNew.InterchangedIndustryUnloader`, and `AlinasMapMod.PaxStationComponent` are normalized at load time for compatibility, but new JSON should use the FUSE names above so it does not depend on external mod assemblies.

Compatibility-converted industry fragments may set `operations.industries.<id>.mergeComponents = true` and component entries may set `partial = true`. In that mode FUSE preserves runtime components omitted from the fragment and patches matching components in place. Partial `trackSpanIds` append to existing spans by default; `trackSpanPatch` can preserve legacy list operations such as `replace`, `prepend`, `append`, `insert`, and `remove`.

Passenger stop components can carry timetable metadata:

```
{
  "type": "passengerStop",
  "name": "Murphy Passenger Stop",
  "trackSpanIds": ["murphy:span:depot"],
  "loadId": "passengers",
  "passengerStopId": "murphy:depot",
  "timetableCode": "MUR",
  "basePopulation": 120,
  "branch": "Murphy Branch"
}
```

Map labels support normal text labels and speed-limit badges. Legacy converted labels such as "15 MPH" are auto-detected at runtime, but new data should use the explicit style:

```
{
  "text": "15",
  "style": "speedLimit",
  "speedLimitMph": 15,
  "position": { "x": 44, "y": 0, "z": 2 }
}
```

Map masks default to FUSE's built-in terrain/object behavior:

- `MaskName.Object`
- cut trees enabled
- terrain height blending disabled

That default is intentional because the simplified FUSE schema stores mask shape, not the full base-game modifier matrix.

Map tile overlays mount additional `tile_XXX_YYY.data` files into Railroader's existing `MapStore`:

- `directory` must match `MapManager.directoryName`, for example `BushnellWhittier`
- `sourceFolder` usually points at a package-relative folder such as `Maps/BushnellWhittier`
- `priority` resolves overlaps when multiple packages provide the same tile; higher numbers win

FUSE treats these as overlays on top of the base game's `StreamingAssets/Maps/<directory>` store. That means a converted tile package does not need to replace `Map.json`; it only needs the extra `.data` tiles it wants to contribute.

Runtime Notes

- `tracks.areas` are applied at runtime and are used as preferred parents for FUSE-created industries. Area and industry order values are also used when rebuilding company-window location ordering.
- `operations.loads` are applied at runtime by creating or updating `CarPrototypeLibrary.instance.opsLoads` entries. Use `units`, `density`, `unitWeightInPounds`, `importable`, `payPerQuantity`, and `costPerUnit` when the custom load needs full behavior parity with legacy mod data. Custom load mods may use `fields` for reflection-bound field/property values on the runtime `Load` object.
- `world.mapMasks` are applied at runtime using the default behavior above.
- `world.waterSurfaces` creates editable polygonal lake planes. Supply at least three world-space points; `sourceLakePath` reuses a specific stock lake's material/profile, while `materialName` selects a loaded material explicitly. `lockHeight`, `snapToTerrain`, `uvScale`, `triangleDensity`, `maximumTriangleArea`, `yOffset`, and `enableCollider` control generation. This native-only contract is intentionally not projected into RailLoader JSON.
- `world.telegraphPoles` are applied at runtime by generating pole instances along the provided point path at the requested spacing.
- `world.telegraphPoleMovements` are applied at runtime by translating existing base-game telegraph pole graph nodes by index. FUSE forces the telegraph manager to refresh when possible.
- `world.spawnPoints` are applied at runtime by creating or updating `Character.SpawnPoint` components under a FUSE-owned world root.

- `world.scenery.*.anchorSpanIds` makes the scenery track-bound at apply time. Missing spans produce warnings and the object falls back to its explicit transform if no anchor resolves.
- `world.mapLabels` are applied at runtime as vanilla `MapLabel` clones. Labels with `style: "speedLimit"` are rendered as circled speed signs.
- `world.mapTiles` are mounted into the live `MapStore` at map-load time and can override individual tile coordinates without copying files into `StreamingAssets`.
- `world.sceneClones` are applied at runtime by cloning an existing scene object path or retargeting an existing object hierarchy. `localPosition`, `localRotation`, and `localScale` are optional; when omitted or null, FUSE now preserves the existing transform value instead of forcing zero or one. FUSE also strips unsupported mover components such as `PhysicsMover` from these clones so static visual copies do not register live physics controllers by accident.
- `world.removals` are applied before new world objects are created. Removal IDs may be FUSE-created object IDs or full scene paths such as `world/Large Scenery/Sylva/Road (1)`.

Validation that depends on Railroader runtime state, such as whether a prefab exists or whether a base-game passenger stop ID is valid, belongs in FUSE's validation layer rather than this JSON Schema.

UMM Package Layout

Recommended data package layout:

```
MurphyBranch/
  Info.json
  track.fuse.json
  operations.fuse.json
  world.fuse.json
  progression.fuse.json
  Maps/
    BushnellWhittier/
      tile_003_015.data
      tile_003_016.data
  Assets/
    Bundle
    Catalog.json
```

`Info.json` stays in Unity Mod Manager's normal PascalCase style. FUSE data files stay in lower camelCase JSON style. FUSE discovers packages by reading `Info.json`, checking `Requirements` or `LoadAfter` for FUSE, then loading `FuseDataFile` or each entry in `FuseDataFiles`. `FuseDataFiles` is preferred for converted packages that keep one file per source concern:

```
{
  "Requirements": [{ "Id": "FUSE", "NotBefore": "1.0.0" }],
  "LoadAfter": ["FUSE"],
  "FuseLoadPriority": 100,
  "FuseLoadAfter": ["Shared.Track.Package"],
  "FuseLoadBefore": [],
  "FuseDataFiles": [
    "track.fuse.json",
    "operations.fuse.json",
    "world.fuse.json"
  ]
}
```

FUSE-specific package ordering uses `FuseLoadPriority`, `FuseLoadAfter`, and `FuseLoadBefore`. Lower priority values load earlier; packages with the same priority fall back to dependency order and then package id. Dependency cycles are reported in the log and the involved packages fall back to priority/name order.

If explicit data-file entries are missing, FUSE falls back to the first `.bson` file in the package root, then an eligible root `.json` definition file. Files named `Info.json`, `Definition.json`, `Catalog.json`, `Definitions.json`, and `conversion-report.json` are ignored; `.fuse.json` files are preferred before other eligible `.json` files.

Asset-pack-only packages can expose existing Railroader `AssetPack` runtime stores with `FuseAssetPacks`. FUSE registers these folders as direct prefab stores from the mod directory by default; the old `LocalLow` mirror path is an opt-in compatibility fallback, not the normal load path:

```
{
  "Requirements": ["FUSE"],
  "LoadAfter": ["FUSE"],
  "FuseAssetPacks": ["SCAssetPacks"]
}
```

Track nodes and segments also preserve the newer graph structure metadata while remaining compatible with released Railroader builds that expose the older four-way style enum:

```
{
  "tracks": {
    "nodes": {
      "murphy:n:diamond-west": {
        "position": { "x": 10, "y": 2, "z": 20 },
        "rotation": { "x": 0, "y": 90, "z": 0 },
        "isDiamond": true
      }
    },
    "segments": {
      "murphy:s:steel-yard-bridge": {
        "startNodeId": "murphy:n:001",
        "endNodeId": "murphy:n:002",
        "style": "bridge",
        "bridgeSupportsSteel": true,
        "yard": true
      }
    }
  }
}
```

`isDiamond` marks a graph crossing node. `bridgeSupportsSteel` selects the steel-support bridge variant, and `yard` is independent of bridge/tunnel structure on game builds that expose combinable flags. Existing style values (`standard`, `bridge`, `tunnel`, `yard`) remain valid. Partial legacy overlays can set `preserveBridgeSupportsSteel` and `preserveYard` just as they already use `preserveStyle`.

`Catalog.json` identifies a runtime store. A local `Bundle` is not required for a definitions-only catalog whose asset references resolve to another store.

Use `FuseDefinitionOverrides` when a package intentionally supplies `Definitions.json` for an existing store without supplying a new catalog/bundle:

```
{
  "Requirements": ["FUSE"],
  "FuseDefinitionOverrides": [
    {
      "StoreIdentifier": "fm-flatcar03",
      "Path": "DefinitionOverrides/fm-flatcar03/Definitions.json"
    }
  ]
}
```

Paths are package-relative and cannot escape the package directory. A string entry is also accepted and infers `StoreIdentifier` from its parent folder.

Converter

Official drag-and-drop converter for legacy Railroader data mods.

> **Data only:** this tool does not convert DLLs or code mods. It can convert > recognized JSON/data from a mixed package, but compiled behavior must remain an > installed compatibility-hosted mod or be independently reimplemented. Use > Install, Convert, Or Author? if you are unsure.

Download

Prebuilt binaries are published on the GitHub Releases page rather than tracked in this repo:

- Windows exe: FUSE-Converter.exe
- Portable Python zipapp: FUSEConvertFolder.pyz

All releases live at <<https://github.com/F-U-S-E-E/FuseDevelopmentGroup/releases>>.

Both tools are attached to the mod release (mod-v* tag), so latest/download resolves to the converter built alongside the FUSE build you are running.

Quick Use

Drag any of these onto ConvertToFUSE.cmd at the repo root:

- a legacy route / track mod folder
- a legacy route / track mod .zip
- a legacy horn / whistle / bell folder or .zip
- a single legacy JSON data file

ConvertToFUSE.cmd launches the official FUSE converter.

Drag asset packs, Alina map-tile packages, native FUSE packages, and compatible code mods onto FUSE-Installer.exe instead. The converter identifies those inputs and explains the correct install path; it does not make empty wrapper packages for them.

The repository also includes the FUSE converter icon at:

tools/assets/fuse_converter.ico

Building The EXE

From the repo root:

```
powershell -ExecutionPolicy Bypass -File .\tools\build_converter_exe.ps1 -InstallPyInstaller
```

That builds:

dist/FUSE-Converter.exe

The dist/ folder is gitignored — locally-built binaries stay on your machine. Published builds are uploaded to GitHub Releases.

The generated exe uses the FUSE logo from tools/assets/fuse_converter.ico. If PyInstaller is already installed, you can omit -InstallPyInstaller.

By default the converter writes to:

```
C:\Steam\steamapps\common\Railroader\Mods
```

If that folder does not exist, it writes to converted under the current working directory.

Command Line

```
python tools\fuse_converter.py "C:\Path\To\LegacyMod" --out "C:\Steam\steamapps\common\Railroader\Mods" --clean
```

fuse-convert (.NET CLI)

FUSE.ConverterCli builds the fuse-convert binary: the C# converter (FUSE.Converter) plus the FUSE.Core validator in one convert → validate → report pipeline. Every validation finding is rendered with a fix-hint from the catalog embedded in FUSE.Core — what's wrong, why it matters, and exactly how to fix it.

```
dotnet run --project FUSE.ConverterCli -- "C:\Path\To\LegacyMod" --out "C:\Steam\steamapps\common\Railroader\Mods" --clean
```

```
fuse-convert <inputs...> [--out <dir>] [--kind auto|route|audio] [--clean] [--batch]
                    [--no-validate] [--strict] [--format console|json|markdown|all] [--quiet]
```

Option	Meaning
--out <dir>	Output root; each mod converts into <dir>\<ModFolder>. FUSE. Default: .\FUSEConverted.
--kind <kind>	Force a package kind instead of auto-detection.
--clean	Replace an existing . FUSE output folder.
--batch	Treat each input as a container. Convert JSON packages and report the correct install guidance for code, asset, tile, or native packages.
--no-validate	Skip validation (it is on by default).
--strict	Validation warnings also fail the run (exit 2), not just errors.
--format <fmt>	console (default) prints to stdout; json / markdown also write reports. Successful reports live in the output package; failed/unsupported reports live under <out>_conversion-reports.
--quiet	Suppress per-diagnostic output; print only summaries.

Exit codes (CI can gate conversion and validation separately):

Code	Meaning
0	Success — validation warnings allowed unless --strict.
1	Conversion failed.
2	Validation errors (or --strict with validation warnings).
64	Usage error.

Unlike the Python tool, fuse-convert takes mod **folders** only — extract a zip (or place a bare JSON in a mod folder) before converting.

Linux / Folder Batch Mode

For Linux or anyone who wants to convert a whole folder at once:

```
python3 tools/fuse_converter.py --batch "/path/to/legacy-mod-folder"
```

That recursively scans the folder for recognized legacy mod folders, zip files, and data JSON files, then writes output to:

/path/to/legacy-mod-folder/FUSEConverted

Or invoke the folder converter directly:

```
python3 tools/fuse_convert_folder.py "/path/to/legacy-mod-folder"
```

To build a single portable Python archive:

```
python3 tools/build_folder_converter_pyz.py
```

That creates:

dist/FUSEConvertFolder.pyz

Drop FUSEConvertFolder.pyz into a folder of legacy mods and run:

```
python3 FUSEConvertFolder.pyz
```

It converts every recognized child folder, zip, and data JSON into FUSEConverted. Wrapper folders such as Mods are traversed automatically, and zip files that contain multiple mods produce one .FUSE output per mod.

Useful options:

Option	Meaning
--out <folder>	Output root for generated .FUSE folders.
--clean	Replace an existing generated .FUSE output folder under the output root.
--kind route	Force route/track/data conversion.
--kind audio	Force horn/whistle/bell conversion.
--batch	Recursively convert every recognized child mod, zip, or JSON in the input folder.

Outputs

Each converted package gets:

- Info.json
- one or more *.fuse.json files, or audio.fuse.json
- copied audio files and asset folders carried by a converted route package when needed
- conversion-report.json
- conversion-report.md

The report is the important bit. It records:

- detected package type
- generated files
- object counts
- warnings for unsupported or lossy legacy concepts
- errors if conversion failed

Supported Inputs

Legacy input	Output
Track / route JSON folders	One FUSE data file per source JSON, preserving file-per-concern structure.
Single route JSON file	One standalone FUSE package fragment.
Horn / whistle / bell packs	FUSE audio package with copied audio files.
Zip files	Extracted to a temporary folder, detected, converted, then cleaned up.

Asset-pack-only, Alina map-tile, native FUSE, and compiled-code packages are not conversion inputs. Install the original package with the FUSE installer. A mixed legacy package may still convert its recognized top-level JSON data; its DLL behavior remains unconverted and is called out in the report.

Known Lossy Areas

The converter warns when it sees concepts that still need manual verification:

- legacy formulaic formula
- progression interchangeTransfers
- unknown spline/object handlers
- unknown industry component types
- script binaries such as .dll / .pdb

Warnings do not always mean the package is unusable. They mean the conversion needs a look before calling it verified.

Unsupported-Feature Policy

The converter should classify unsupported content instead of silently dropping it.

Outcome	Meaning
converted	The source concept maps directly to supported FUSE schema/runtime behavior.
repaired	The converter changed invalid legacy data into valid equivalent FUSE data and reported the repair.
preserved	The source data is kept in extensions or package files for future/manual work, but does not have full runtime behavior yet.
dependency-required	The source entry can work when another package/asset/component dependency is installed.
unresolved	FUSE could not resolve a reference from the source data; the output keeps enough context for manual repair where possible.

Outcome	Meaning
unsupported	FUSE intentionally does not support the legacy behavior. The report must name the source file and concept.
error	Conversion failed for that input and the output should not be treated as usable.

Current intentional unsupported/deferred areas:

- signals
- normal three-way switch authoring, because Railroader does not support it as a standard graph switch
- arbitrary legacy script logic from .dll files
- rolling stock/car/locomotive packages outside FUSE audio definitions
- unknown custom component types when their owning assembly is not installed or not converted to a FUSE component package

The presence of a .dll is never a successful code conversion. The report must keep that binary visible as manual/unsupported work even when the package's JSON converted successfully.

Unsupported entries must remain visible in `conversion-report.json` and `conversion-report.md`. A clean conversion is allowed to have `info` entries, but a supported package should not rely on hidden unsupported runtime behavior.

External Editor

The FUSE External Editor is a standalone desktop application for building FUSE packages outside the game — terrain, track graph, and scenery, with a live link that pushes changes into a running Railroader session.

It ships and versions independently of the mod. The in-game editor is not part of this release.

Download

Self-contained per-OS bundles are attached to each `externaleditor-v*` release on the Releases page:

- `FUSE.ExternalEditor-v<ver>-win-x64.zip`
- `FUSE.ExternalEditor-v<ver>-linux-x64.zip`
- `FUSE.ExternalEditor-v<ver>-osx-x64.zip`

The bundles are self-contained — no .NET runtime install required. Extract and run `FUSE.ExternalEditor`.

The editor is game-free: it does not need Railroader installed to run. You only need the game for the Live Bridge and for testing what you build.

Editor and mod version numbers are independent. A given editor build is not tied to a matching mod build.

First Run

Point the editor at your game's Mods folder with **Set Mods....** This is what lets it save packages where the game will find them and lets the Live Bridge locate a running session. You only need to do it once.

Then either:

- **New Mod...** to start an empty package,
- **Open FUSE Mod...** to load an existing *.FUSE package, or
- **Convert Legacy...** to import a legacy mod directly, without running the standalone converter first.

Save Mod... writes the package back out.

The Workspace

A central viewport with docked panels:

Panel	What it shows
Entities	The package's object tree — nodes, segments, scenery, and the rest
Selection	Properties of what is currently selected
Track Graph	Graph-level view of nodes and connections
Profile	Elevation profile along the selection, with a running cut/fill figure in metres
Calculator	An expression box for arithmetic without leaving the app
Live Bridge	Connection state and the push control

Undo and **Redo** apply across editing operations.

Track Tools

Pick a tool from the toolbar, then work in the viewport.

Tool	Purpose
Select	Pick nodes and segments
Move	Reposition the selection
Add Node	Place a new graph node
Connect	Join two nodes with a segment
Curve	Curve a connection
Fit Arc	Fit a circular arc through the selection
Turnout	Place a turnout
Wye	Place a wye
Delete	Remove the selection
Measure	Measure distance in the viewport

Scenery places scenery objects.

Three-way switches are not available, because Railroader does not support them as standard graph switches.

Terrain

Open Terrain... loads terrain to edit; **Save Terrain** writes it back.

Sculpting tools share a **Brush** with **Radius** and **Strength** controls:

Tool	Effect
Raise	Raise or lower ground
Flatten	Level toward a target height
Smooth	Soften sharp transitions
Erode	Simulated erosion
Noise	Add height variation
Paint Terrain	Paint surface types

Watch the **Profile** panel's cut/fill readout while grading — it is the fastest way to tell whether an alignment is buildable or whether you are moving an unreasonable amount of earth.

Generating Terrain

Generate builds terrain from real-world data for a given location and dimensions (gx, gy, w, h, and height). Real-world elevation sources need a Mapbox token, entered in the generation panel.

Additional layers:

- **NLCD** — land cover classification
- **Veg** — vegetation
- **Water** — water bodies

- **Hillshade** — shaded relief, for reading terrain shape in the viewport

OSM Overlay

Fetch OSM pulls OpenStreetMap data for the current area and **OSM** toggles the overlay. Use it to trace real alignments — roads, existing rail, and water features — as a reference while placing track.

Both OSM and Mapbox data come from third-party services under their own licences and terms. Check those terms before redistributing anything derived from them in a published package.

Live Bridge

The Live Bridge pushes edits into a running game session without a restart.

Requires the `FUSE.LiveBridge` **mod**, a separate zip attached to the mod release. Install it in `Railroader/Mods` alongside `FUSE`.

With the game running and a map loaded, **Push to Game** sends the current package for reload. **Refresh** re-reads the connection state.

The bridge is file-based: the editor writes a reload command into the package folder and the in-game mod picks it up, reporting a heartbeat back. The editor classifies the connection as stale when the heartbeat is more than five seconds old — if the panel shows stale, confirm the game is running, a map is loaded, and the LiveBridge mod is enabled.

The bridge is a development convenience. Verify a package by loading it normally before publishing.

Publishing A Package

- 1 **Save Mod...** into your Mods folder.
- 2 Load it in-game normally, without the bridge.
- 3 Run `/fuse.report` and `/fuse.validate <modId>` to confirm it is clean.
- 4 Check `/fuse.conflicts` if other packages touch the same route.

See `PACKAGE_AUTHOR_GUIDE.md` for the authoring contract that governs ids, dependencies, and optional references.

Related

- `PACKAGE_AUTHOR_GUIDE.md` — the authoring contract
- `schemas/FUSE_JSON_SCHEMA.md` — the schema the editor writes
- `FUSE_CONVERTER.md` — batch and CLI conversion
- `CONSOLE_COMMANDS.md` — verifying in-game

Migrating From Legacy Mods

FUSE replaces the legacy Railroader modding stack. This guide covers moving an existing install — Railroader, Strange Customs, ConfusingSupplements, For Your Convenience, or Alina's Map Mod — over to FUSE packages.

If you are setting up a fresh install with no legacy mods, you do not need this page. See `GETTING_STARTED.md`.

What Changes

FUSE can host compatible RailLoader plugins and translate supported legacy data in memory. Converting a data mod to native FUSE is still recommended for authors because it produces validation and conversion reports, but players do not have to convert every compatible package before testing it.

What carries over:

Legacy content	Status under FUSE
Track / route JSON	Converted, one FUSE data file per source JSON
Horn / whistle / bell packs	Converted, audio files copied
Strange Customs asset packs	Installed directly; FUSE discovers supported legacy asset packs
Alina map-tile packages	Installed directly; FUSE's Alina compatibility reads the tile data
World scenery, scene clones, map masks, splines	Converted
Industries, loaders, stations, team/repair tracks	Converted
Progression sections and delivery phases	Converted

What does not carry over:

- **Unknown arbitrary script behavior.** The converter cannot translate DLL logic. FUSE independently implements the legacy host APIs used by supported RailLoader plugins, but a plugin using an unimplemented API is isolated and reported instead of being silently converted.
- **Rolling stock, locomotive, and car mods**, except audio definitions.
- **Signals.**
- **Three-way switches** — Railroader does not support them as standard graph switches, so legacy content authoring one has an authoring problem that predates FUSE.

The Critical Rule

Do not run a legacy route and its converted FUSE equivalent at the same time.

Both stacks will try to own the same object ids, and you get duplicate track, duplicate industries, or duplicate buildings. `/fuse.conflicts` reports exactly this situation, and it is the single most common self-inflicted problem when migrating.

The same applies to the loaders themselves. FUSE detects a leftover `Railloader.dll`, `Railloader.Injector.dll`, or `Railloader.Interchange.dll` in the install and warns about it, both on disk and among loaded assemblies. Take the warning seriously — it means two loaders are live at once.

Loading both is supported only as a deliberate conflict test.

Migration Steps

1. Back up first

Back up your saves and your entire Mods folder before touching anything. This is the step people skip and regret.

2. Inventory what you have

List your legacy mods and sort them into three groups:

- **Convertible data mods** (routes, scenery, industries, supported audio JSON) — these convert.
- **Direct-install data** (asset packs and Alina map tiles) — install the original package through the FUSE installer; do not run it through the converter.
- **Script mods** (.dll logic) — these do not convert; check whether a FUSE-native equivalent exists.
- **Rolling stock** — out of scope except audio.

3. Convert the data mods

Point the converter at each legacy mod folder. The drag-and-drop route:

```
.\FUSE-Converter.exe
```

Or the .NET CLI, which converts and validates in one pass:

```
fuse-convert "C:\Path\To\LegacyMod" --out "C:\Steam\steamapps\common\Railroader\Mods" --format all
```

--batch converts an entire folder of legacy mods at once. Full options are in FUSE_CONVERTER.md.

4. Read the conversion report

Every converted package gets `conversion-report.json` and `conversion-report.md`. Failed or direct-install inputs write their reports under the output root's `_conversion-reports` folder so the report cannot be mistaken for a package. **Read them.** The converter classifies each source concept rather than silently dropping it:

Outcome	What it means for you
converted	Maps directly to FUSE. Nothing to do.
repaired	Invalid legacy data was fixed. Worth a look, but generally fine.
preserved	Kept in the package but without full runtime behavior yet.
dependency-required	Works once you install the named dependency.
unresolved	A reference could not be resolved — needs manual repair.
unsupported	FUSE intentionally does not support this. Named in the report.
error	Conversion failed. Do not ship or rely on this package.

Warnings do not automatically mean a package is broken. They mean the conversion needs a look before you call it verified.

The lossy areas that most often need manual verification: legacy formulaic formula values, progression interchangeTransfers, unknown spline or object handlers, unknown industry component types, and script binaries.

5. Remove the legacy versions

Once the converted package is in place, remove or disable the legacy original. Both cannot be active.

Remove the legacy loader itself only after every data mod you care about has been converted and verified — you may want to go back.

6. Install dependencies

Converted route packages frequently depend on asset packs. Install them as real asset packs. Do not work around a missing asset by aliasing it to a different name; if the correct pack exists, install it. `/fuse.assets` lists what FUSE actually found.

7. Verify in game

Load a map and check:

```
/fuse.report  
/fuse.loaded  
/fuse.conflicts
```

A clean migration shows zero faults and zero conflicts. Then look at the world — location list, progression visibility, and terrain masks are the things that look wrong in ways a report will not catch.

Per-Mod Notes

Railloader

The legacy loader. FUSE reads Railloader's package metadata directly, including `RailLoadPriority`, `RailLoadAfter`, and `RailLoadBefore` for load ordering, so ordering relationships you already established carry across.

Remove `Railloader.dll`, `Railloader.Injector.dll`, and `Railloader.Interchange.dll` once you have migrated — FUSE warns while they remain.

Strange Customs

Asset packs install directly and remain available to native or converted route packages. Mixin files convert through the `mixinto` mechanism; a missing `mixinto` requirement skips only that fragment rather than faulting the whole stack.

ConfusingSupplements / For Your Convenience

Data content converts. Any script-driven behavior does not — check the conversion report for what was classified unsupported.

Alina's Map Mod

Map content converts. As with any route, do not run the legacy version alongside the converted one.

When To Reconvert

Reconvert a package when:

- The FUSE converter version changes.
- The schema version changes.
- A converter repair changed from warning-only to an actual runtime fix.
- The legacy package was updated or redownloaded.

If Something Goes Wrong

Restore your backup — that is what it is for. Then narrow the problem down to a single package by disabling packages until the symptom disappears, and file a report with the conversion report for that one package attached.

TROUBLESHOOTING.md maps symptoms to the diagnostic command that explains them.

Related

- FUSE_CONVERTER.md — full converter reference
- GETTING_STARTED.md — installing FUSE itself
- KNOWN_ISSUES.md — known legacy interaction problems

Architecture

An orientation for contributors: what each project is, how a package gets from disk into the running game, and where the seams are.

For build setup, see ../CONTRIBUTING.md.

Projects

The solution splits along one main axis: **does this code need the game?**

Game-side (net48, Unity + UMM)

Project	Role
FUSE	The mod itself. Loads packages, applies them to the running world, owns the runtime UI and console commands.
FUSE.Editor	In-game editor. Also the entry project CI builds — it pulls in FUSE.
FUSE.LiveBridge	Optional in-game half of the hot-reload bridge. Ships as its own zip.
FUSE.TestBridge	In-game hooks for the golden-master test harness.
FUSE.Tests	xUnit 2.x tests. net48, pinned there deliberately.

These target net48 because that is what the Unity runtime hosts. They resolve Unity and Railroader types from the game install, or from the checked-in lib/refs reference assemblies when no game is present.

Game-free (net10.0)

Project	Role
FUSE.Core	The shared model, schema, validation, serialization, migrations, and geometry. Multi-targets net48;net10.0 so both worlds use one implementation.
FUSE.Converter	Legacy → FUSE conversion. Also multi-targets.
FUSE.ConverterCli	The fuse-convert binary: convert → validate → report.
FUSE.ExternalEditor	Standalone Avalonia desktop editor.
FUSE.LiveHarness / FUSE.TestCli	Test tooling.
FUSE.Core.Tests, FUSE.LiveHarness.Tests, FUSE.ExternalEditor.UiTests	Tests for the game-free half.

FUSE.Core's multi-targeting is the load-bearing decision here. Validation and schema logic run identically in the game, in the converter, and in the external editor because they are literally the same code, not three implementations that drift.

Where The Testable Code Lives

Anything reachable without UnityEngine can be unit tested: validation, serialization, dependency resolution, registry conflict logic, pure data conversion. That is most of FUSE.Core and FUSE.Converter.

Anything touching game or Unity types cannot, and is verified by the in-game golden-master harness instead. When adding logic, consider whether it can live on the testable side of that line — usually it can.

The Loading Pipeline

FUSE/Loading holds most of this.

1. Discovery

FuseDataPackageDiscovery walks Railroader/Mods for FUSE packages; FuseDefinitionFileDiscovery finds the *.fuse.json files inside each one. FuseEarlyLoader handles work that must happen before the game gets far into startup.

UMM's enabled checkbox is honored here — a package UMM can see and has disabled is marked disabled and its data files are not read.

2. Legacy conversion

FuseLegacyDataConverter (split across .World, .Progression, .Components, .Audio, .Json, .StringPatches) translates legacy formats in memory. FuseLegacyJsonPatch and FuseLegacyContainerMixintoRegistry handle legacy patch and mixinto semantics.

This is the same conceptual work as the standalone converter, applied at load time for compatibility rather than producing files on disk.

3. Validation and requirements

FuseModLoader.Validation runs the FUSE.Core validator. FuseModRequirementResolver resolves dependencies and load order, honoring RailLoadPriority, RailLoadAfter, and RailLoadBefore.

4. Apply

FuseModLoader.Apply writes definitions into the live world; FuseModLoader.TrackMerging merges track graph changes into the existing graph.

FuseApplyTransaction and FuseRegistryTransaction make this recoverable rather than a partial mutation on failure.

5. Ownership and conflicts

FuseRegistry records which package claims which object, keyed by FuseClaimKind. Two packages claiming the same id produces a FuseRegistryConflict — what /fuse.conflicts reports.

This is the mechanism behind the "never run a legacy route and its converted equivalent together" rule: both claim the same ids.

6. Reporting

FuseLoadReport accumulates the outcome; FusePackageFaultRegistry records per-package faults. This is what /fuse.report and the startup toast render.

A faulted package is isolated — PackageApplyOutcome carries the per-package result so one failure does not take down unrelated packages.

Design Rules

Three conventions explain most of what looks unusual in this codebase.

Isolate third-party failure

One broken package must not crash the host. Broad catch (`Exception`) in the loading path is intentional design, which is why CA1031 is excluded from the analyzer set repo-wide. Catch, record against the package, keep going.

`FusePatchResilience` applies the same idea to Harmony patches: a patch that fails to apply is recorded and skipped, not fatal. `/fuse.patches` lists both outcomes.

Report, do not hide

FUSE repairs what it can safely repair and reports everything else. Unsupported graph shapes, missing hard dependencies, invalid spans, and missing component assemblies get reported rather than silently dropped. The converter's outcome classification (`converted` / `repaired` / `preserved` / `dependency-required` / `unresolved` / `unsupported` / `error`) is the same principle in the offline tool.

Diagnostics name the package

A warning should identify the package id, operation, object id, and field. The console commands and the various registries exist so that a user-reported problem can be traced to a specific package and object without a debugger.

Runtime Services

`FUSE/Runtime/Lifecycle` holds the services that run during play rather than at load: deferred scenery activation, terrain rebaking, texture memory policy under constrained VRAM, unused asset reclamation, frame spike and hitch detection, and the runtime reload/rebind services behind the experimental console commands.

`FUSE/Infrastructure` holds cross-cutting concerns — logging, settings, multiplayer guards, UMM injection, exception attribution to the owning mod, and performance metrics.

UI

`FUSE/Interface/MenuWindow` builds the in-game FUSE menu (Status and Tools tabs);

`FUSE/Interface/Console` registers the `/fuse.*` commands via `Railroader's ConsoleCommandAttribute`.

Adding a command means adding a class with that attribute — and an entry in `CONSOLE_COMMANDS.md`.

Related

- `../CONTRIBUTING.md` — build and contribution process
- `EDITOR_ARCHITECTURE.md` — editor design and target shape
- `../schemas/FUSE_JSON_SCHEMA.md` — the data contract
- `RELEASING.md` — how builds ship